



BiteHouse

Good food. Made fresh.



React Restaurant Ordering Website

Project Documentation

Technology: React, React Router, Vite, JavaScript, CSS

Project Type: Frontend Web Application

Date: September 2026

Table of Contents

1. Introduction	Project overview
2. Application Flowchart	User journey
3. React Topics Used	Concept summary
4. Topic Descriptions and Usage	Implementation details
5. Deployment Steps	GitHub and Netlify
6. Output Screenshots	Application results
7. References	Technical resources

1. Introduction

BiteHouse is a responsive restaurant ordering application developed with React. It allows customers to explore a food menu, find dishes, choose dine-in or takeaway, manage an order, and complete a mock checkout.

The application presents approximately 30 dishes across categories such as burgers, pizza, biryani, pasta, sandwiches, wraps, starters, desserts, and drinks. Every dish includes a name, description, food image, price, rating, availability, and vegetarian or non-vegetarian indicator.

Customer Experience Search, filters, categories, sorting, wishlist, cart, and responsive navigation provide a practical ordering flow.	Order Modes Customers can select Takeaway or Dine In. Dine-in customers can choose Table 1 through Table 15.
Browser Persistence Cart, wishlist, notification requests, order type, and table selection are stored in localStorage.	No Backend Required Menu and stock data are local JavaScript objects, making the project simple to run and deploy.

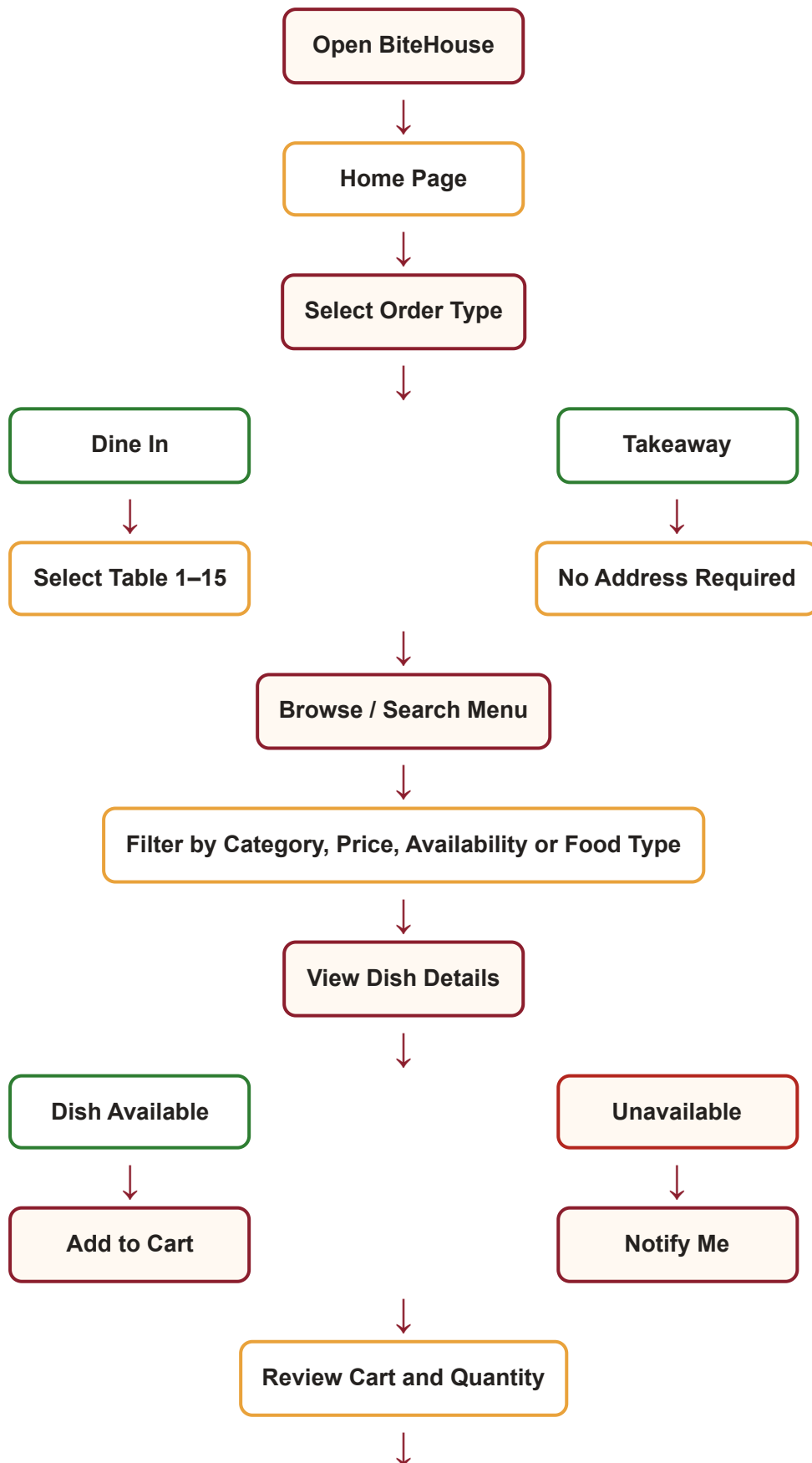
Project Objectives

- Build a complete multi-page restaurant ordering interface with reusable React components.
- Demonstrate state management, routing, forms, validation, filtering, and persistent browser storage.
- Support realistic restaurant workflows without requiring a backend or payment gateway.

- Provide a responsive experience for desktop, tablet, and mobile devices.

2. Application Flowchart

The following flowchart shows the main customer journey from opening BiteHouse through order confirmation.



Checkout and Choose Payment



Order Confirmed: BH-xxxxx

Alternative actions available during the flow include adding or removing wishlist items, contacting the outlet by call or SMS, changing order type, and returning to the menu.

3. React Topics Used

React Topic	Purpose in BiteHouse	Main Location
Functional Components	Build reusable UI sections and pages.	src/components , src/pages
JSX	Describe the interface using HTML-like JavaScript syntax.	All .jsx files
Props	Pass dishes, callbacks, quantities, and display values between components.	ProductCard , ProductGrid , QuantitySelector
useState	Manage interactive values such as search, filters, forms, modals, and mobile menus.	Shop.jsx , Checkout.jsx , Navbar.jsx
useEffect	Synchronize React state with localStorage and URL search parameters.	Context providers, Shop.jsx
useMemo	Compute filtered menu results and detailed cart/wishlist data efficiently.	Shop.jsx , context providers
Context API	Share cart, wishlist, and order settings across routes.	src/context
React Router	Provide client-side navigation and dynamic dish routes.	App.jsx , navigation components
Conditional Rendering	Display available/unavailable actions, empty states, forms, and success screens.	Cards and page components
List Rendering and Keys	Render categories, dishes, cart rows, and table options from arrays.	Menu, Home, Cart, selectors
Controlled Forms	Keep checkout and notification input values in React state.	Checkout.jsx , NotifyModal.jsx
Event Handling	Respond to clicks, submissions, input changes, and keyboard actions.	Throughout the application
Custom Hooks	Expose clean access to shared contexts.	useCart , useWishlist , useOrder
StrictMode	Enable extra development checks for component behavior.	main.jsx

Project Structure

```
src/  
├─ components/ Reusable navigation, cards, filters, modal and controls  
├─ context/ Cart, wishlist and order preference state
```

- └─ data/ Local menu data for 30 dishes
- └─ pages/ Home, Menu, Details, Cart, Wishlist and Checkout
- └─ styles/ Responsive global design system
- └─ utils/ Storage, currency, phone and notification helpers
- └─ App.jsx Route definitions and shared layout
- └─ main.jsx React application entry point

4. React Topic Descriptions and Usage

4.1 Functional Components and JSX

Functional components are JavaScript functions that return JSX. BiteHouse separates the UI into focused units such as `Navbar`, `ProductCard`, `SearchBar`, `NotifyModal`, and complete route pages. JSX keeps the structure close to its dynamic data and event handlers.

Used in: `src/components/*.jsx` and `src/pages/*.jsx`

4.2 Props

Props allow a parent to configure a reusable child. `ProductGrid` receives a filtered `products` array and passes each dish to `ProductCard`. `QuantitySelector` receives `quantity`, `max`, and `onChange`, so the same control can enforce each dish's stock limit.

Used in: `ProductGrid.jsx`, `ProductCard.jsx`, `QuantitySelector.jsx`, `EmptyState.jsx`

4.3 `useState`

The `useState` hook stores values that change while the user interacts with the app. The Menu page tracks search text, selected category, maximum price, sorting, stock-only mode, food type, and the dish selected for notification. Checkout stores customer form values, errors, and the generated order ID.

Used in: `Shop.jsx`, `Home.jsx`, `ProductDetails.jsx`, `Checkout.jsx`, `NotifyModal.jsx`, `Navbar.jsx`

4.4 `useEffect`

`useEffect` performs work after rendering when specified dependencies change. The context providers save cart, wishlist, order type, and table values to `localStorage`. The Menu page also responds to URL search and category parameters. `NotifyModal` registers and cleans up its Escape-key listener.

Used in: `CartContext.jsx`, `WishlistContext.jsx`, `OrderContext.jsx`, `Shop.jsx`, `NotifyModal.jsx`

4.5 `useMemo`

`useMemo` stores a calculated result until its dependencies change. BiteHouse uses it for menu filtering and sorting, for turning stored cart IDs into detailed dish objects, and for building the wishlist's dish collection.

Used in: `Shop.jsx`, `CartContext.jsx`, `WishlistContext.jsx`

4. React Topic Descriptions and Usage (Continued)

4.6 Context API and Custom Hooks

Context shares state without passing it through every intermediate component. CartProvider exposes items, totals, quantity changes, removal, and clearing. WishlistProvider exposes saved dishes and toggle actions. OrderProvider exposes Dine In/Takeaway and table selection. Custom hooks such as `useCart()` provide a simple component-facing API.

Used in: `src/context/CartContext.jsx`, `WishlistContext.jsx`, `OrderContext.jsx`

4.7 React Router

BrowserRouter enables navigation without complete page reloads. Routes map URLs to page components, while Link and NavLink provide navigation. `/product/:id` is a dynamic route; ProductDetails reads the ID with `useParams`. Search navigation uses `useNavigate`, and Checkout uses `Navigate` to prevent access with an empty cart.

Used in: `main.jsx`, `App.jsx`, `Navbar.jsx`, `ProductDetails.jsx`, `Checkout.jsx`

4.8 Conditional Rendering

React expressions select the correct interface for the current state. Available dishes show Add, while unavailable dishes show Notify Me. Empty cart and wishlist states render dedicated messages. Checkout switches between Takeaway customer fields, the Dine In table message, and the final confirmation screen.

Used in: `ProductCard.jsx`, `Cart.jsx`, `Wishlist.jsx`, `Checkout.jsx`

4.9 List Rendering and Keys

JavaScript `map()` converts arrays into components. This is used for dish cards, categories, cart items, payment options, related dishes, and Table 1–15. Stable IDs are used as React keys to help React update lists correctly.

Used in: `ProductGrid.jsx`, `CategoryFilter.jsx`, `Cart.jsx`, `OrderTypeSelector.jsx`

4.10 Controlled Forms and Validation

A controlled input receives its value from state and updates that state through `onChange`. BiteHouse uses this method for checkout and notification forms. Validation checks required Takeaway fields, Indian phone numbers, and duplicate notification requests before data is accepted.

Used in: `Checkout.jsx`, `NotifyModal.jsx`, `SearchBar.jsx`, `storage.js`

4.11 Event Handling

Click handlers add dishes, update quantities, toggle the wishlist, choose order types, and close menus. Submit handlers prevent normal page submission and execute React logic. Change handlers update filters immediately, creating a responsive menu browsing experience.

Used throughout components and pages.

4.12 localStorage Integration

Although localStorage is a browser API rather than a React feature, it is integrated with state initializers and effects. Data is loaded when providers initialize and saved whenever shared state changes.

Notification requests also use localStorage with product-and-phone duplicate detection.

Used in: `storage.js`, context providers, `notifications.js`

5. Deployment Steps

5.1 Prepare and Test Locally

- 1 Install Node.js and open a terminal in the BiteHouse project directory.
- 2 Install dependencies with `npm install`.
- 3 Start the local development server using `npm run dev`.
- 4 Open the Vite address, usually `http://localhost:5173`, and test all routes and controls.
- 5 Create a production build with `npm run build`. Vite writes optimized files to `dist/`.
- 6 Optionally inspect the build using `npm run preview`.

5.2 Push to GitHub

- 1 Initialize Git if required: `git init -b main`.
- 2 Connect the repository: `git remote add origin https://github.com/meghanadh05/Bitehouse.git`.
- 3 Stage and commit: `git add .` followed by `git commit -m "Build BiteHouse app"`.
- 4 Push the main branch using `git push -u origin main`.

5.3 Deploy on Netlify

- 1 Sign in to Netlify and select **Add new site** → **Import an existing project**.
- 2 Choose GitHub and select the **Bitehouse** repository.
- 3 Set the build command to `npm run build`.
- 4 Set the publish directory to `dist`.
- 5 Deploy the site. Netlify installs dependencies and runs the production build automatically.
- 6 Verify direct links such as `/shop` and `/product/9`. The included `public/_redirects` file contains `/* /index.html 200`, allowing React Router routes to work after refresh.

Environment Notes

- No backend service, database, environment variables, or API keys are required.

- Cart and preference data is stored separately in each visitor's browser.
- Payment options are demonstrations only; no financial transaction is performed.

6. Output Screenshots

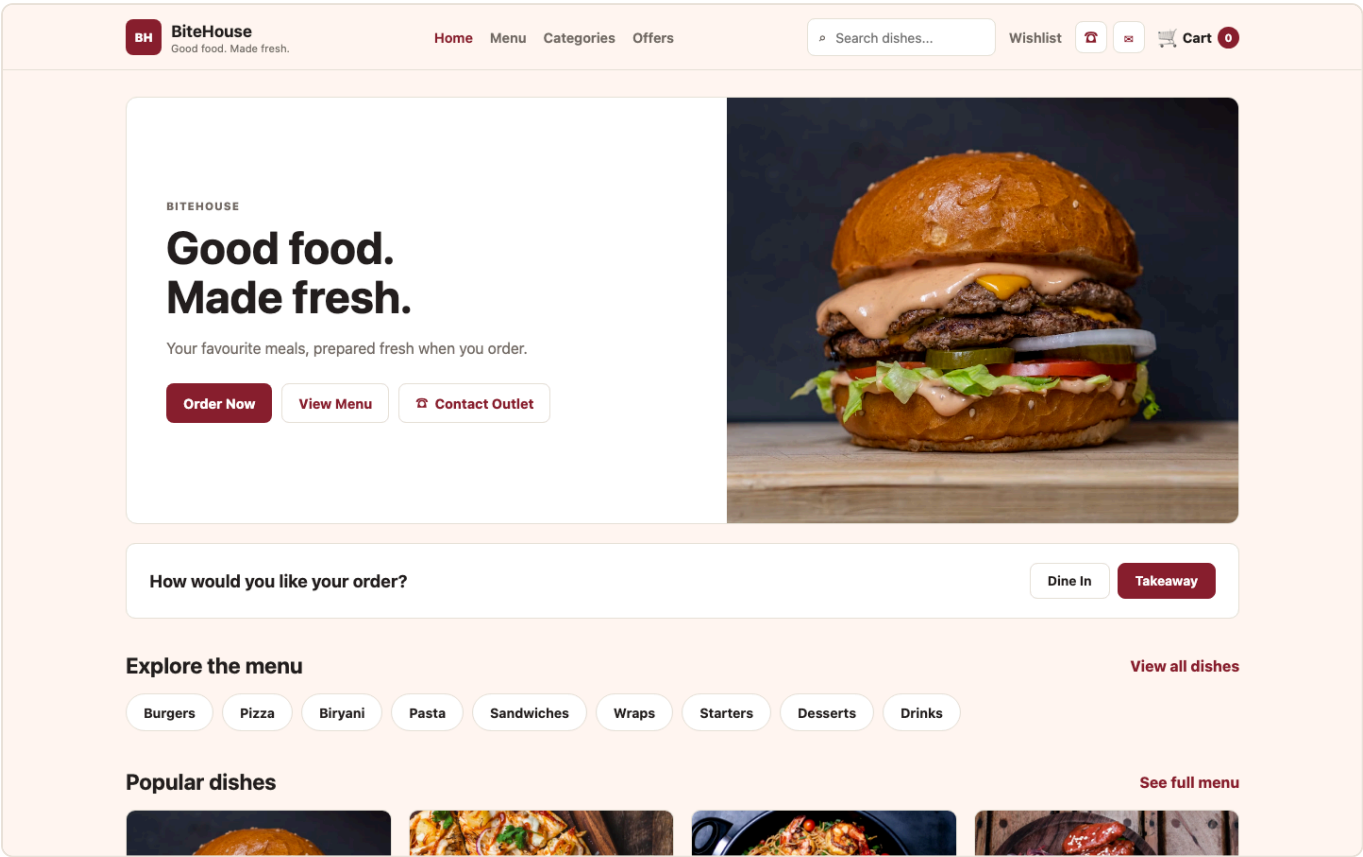


Figure 1: BiteHouse home page with hero, outlet contact action, order type selector, and food categories.

6. Output Screenshots (Continued)

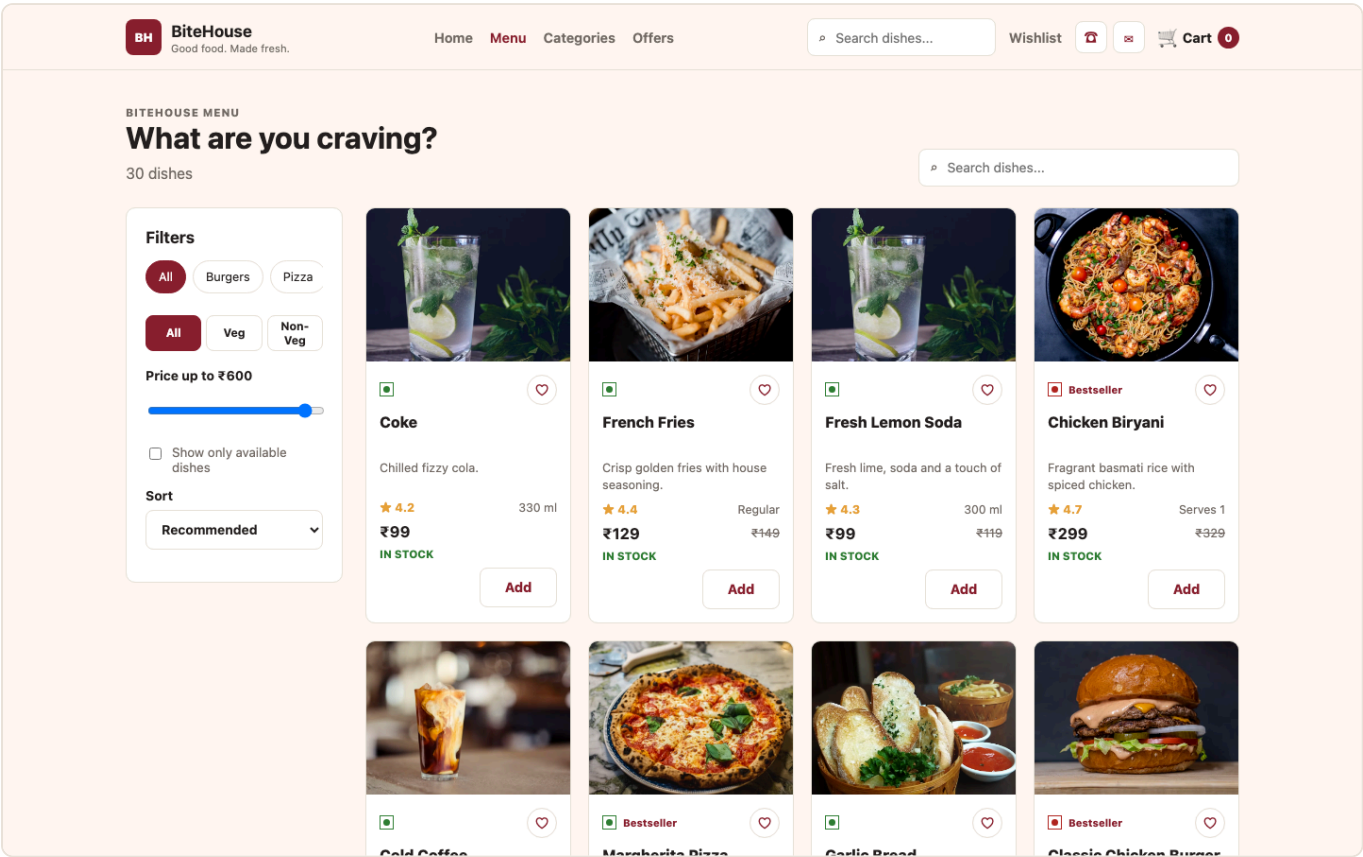


Figure 2: Menu page with search, category tabs, price control, availability, veg/non-veg filters, sorting, and dish cards.

6. Output Screenshots (Continued)

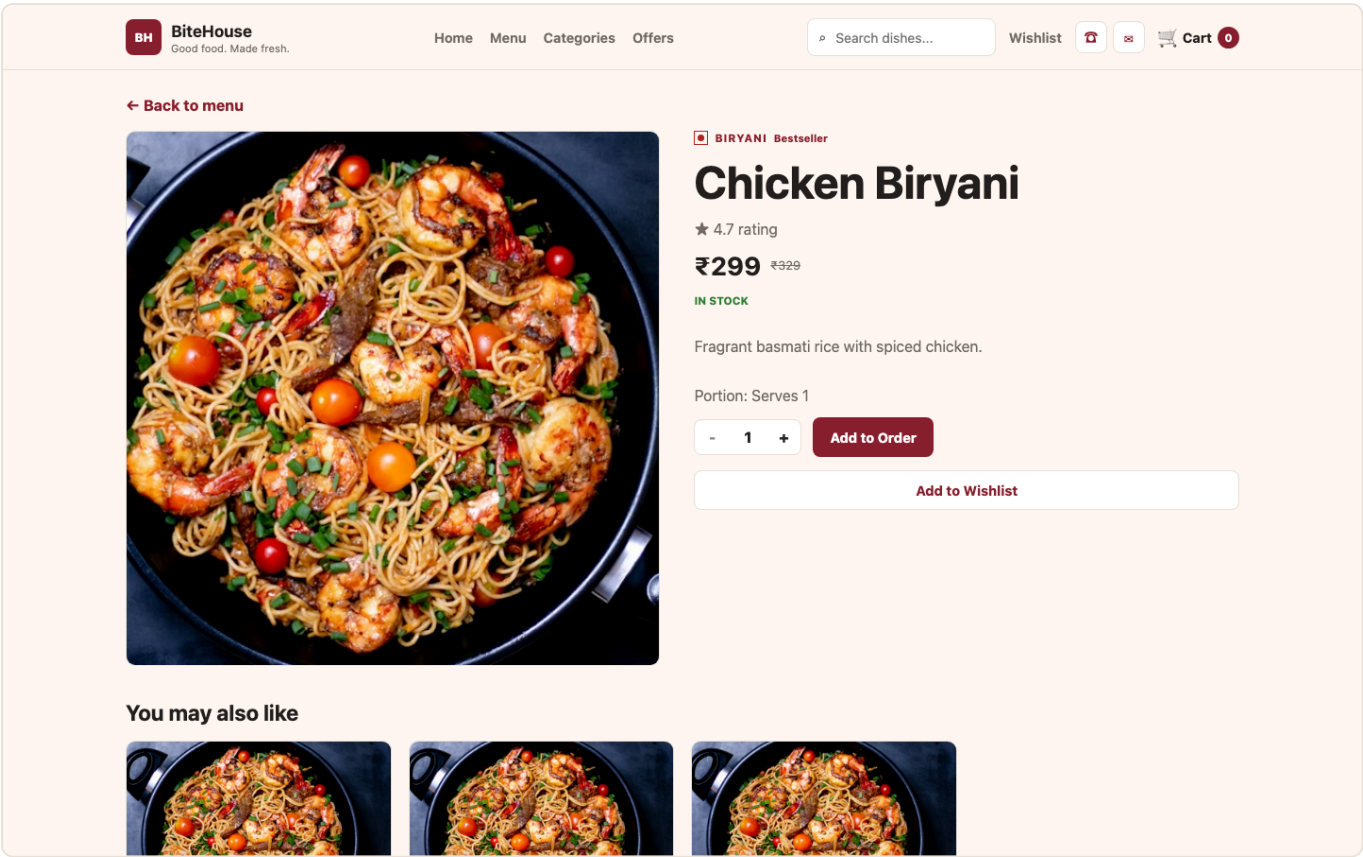


Figure 3: Dish details route showing food type, rating, price, stock status, quantity selector, cart action, and related dishes.

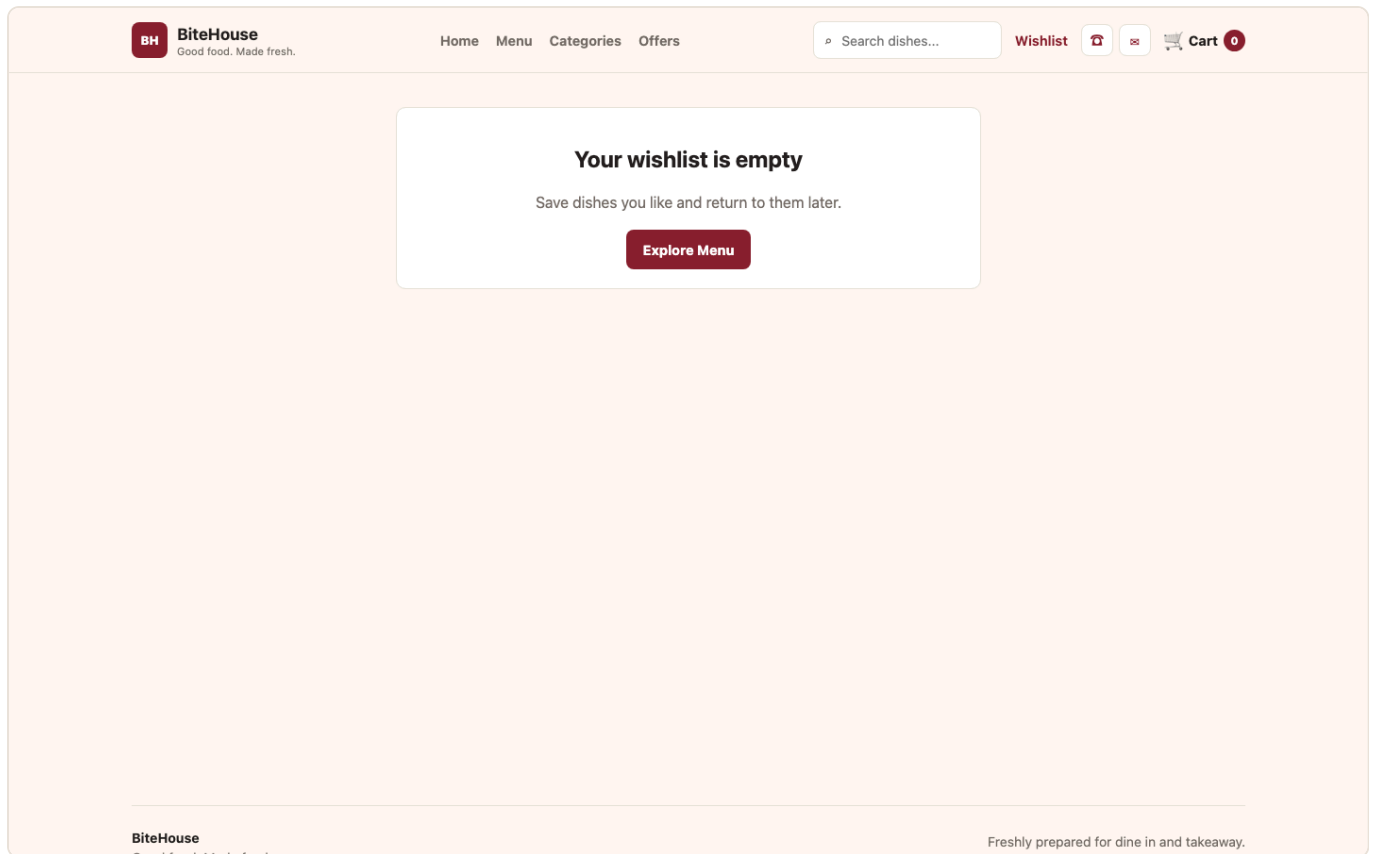


Figure 4: Wishlist empty state with a direct action to explore the menu.

7. References

1. **React Documentation.** Components, JSX, props, state, effects, context, and hooks.
<https://react.dev/>
2. **React Router Documentation.** BrowserRouter, Routes, Route, Link, NavLink, navigation, and URL parameters.
<https://reactrouter.com/>
3. **Vite Documentation.** Development server, React integration, and production builds.
<https://vite.dev/guide/>
4. **MDN Web Docs: Window.localStorage.** Persistent key-value storage in the browser.
<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
5. **MDN Web Docs: HTML Forms.** Input controls, form submission, and client-side validation concepts.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms
6. **Netlify Documentation.** Vite deployment and redirect configuration for single-page applications.
<https://docs.netlify.com/>
7. **Unsplash.** Food photography used in the menu dataset and interface screenshots.
<https://unsplash.com/>
8. **Git Documentation.** Version control, commits, remotes, and pushing to GitHub.
<https://git-scm.com/doc>

Conclusion

BiteHouse demonstrates the main concepts required for a complete React frontend project: component-based design, reusable props, hooks, context-based shared state, client-side routing, form validation, localStorage persistence, responsive CSS, and deployment-ready production builds. The result is a realistic restaurant ordering experience that runs entirely in the browser.